

Engineering reasoning deepening

H.A.R.D. Protocol 0.3 · Public Preview

This is a targeted-deepening lens inside H.A.R.D. It is not a seventh gate, a separate score, a software architecture standard or a claim that one engineer's private thought process can be reconstructed.

Why this lens exists

AI-assisted creation can produce a plausible plan or implementation before the responsible people have explicitly formed the system model that lets them judge it. H.A.R.D. calls this motivating problem **decision compression**: materially different design and engineering decisions can collapse into one generated result.

The review reverses that compression without inventing history:

Decompress -> Model -> Challenge -> Prove -> Decide

- **Decompress**: identify observed consequential choices, their purpose, criteria, alternatives, assumptions and tradeoffs.
- **Model**: externalize only the system concepts that the choice actually depends on.
- **Challenge**: identify a bounded condition that could make the choice or model wrong.
- **Prove**: connect the claim to stage-appropriate evidence at the exact revision.
- **Decide**: a human accepts, revises or leaves the choice pending and names the next bounded step.

The lens does not ask an AI to produce a more persuasive rationale. A generated explanation is a new hypothesis unless retained evidence shows it was part of the historical decision.

When engineering deepening applies

For every important choice, record yes or no and a reason. Deepening is normally warranted when the choice materially depends on one or more of these conditions:

1. Persistent state is created, changed or deleted.
2. An external side effect occurs.
3. The action is difficult or impossible to reverse.
4. A privileged, authorization or tenant boundary is crossed.
5. An external or asynchronous dependency can be slow, unavailable or ambiguous.
6. The same operation can repeat or concurrent operations can interact.
7. Failure can lose money, data or another consequential resource.
8. A material scale, latency, capacity or cost assumption controls the design.

9. A current user promise depends on work labeled deferred or future.
10. More than one plausible source of truth exists.
11. Another condition creates comparable consequence if the implicit model is wrong.

Do not mark deepening required only because code is complex or AI wrote it. Do not mark it unnecessary only because the interface looks simple.

Decision surfaces

Decision surfaces are not mandatory categories to fill for every choice. The reviewer names the kinds that are necessary to understand the specific consequential decision.

Truth

Question: What is true in the system, and where is that truth authoritative?

Record the authoritative record or rule, derived/cached representations and conflict behavior when relevant.

Example: membership state is authoritative in the membership service. A UI badge or cached claim is a representation, not separate authority.

Ownership

Question: Who owns that truth and who is allowed to mutate it?

Ownership can be a person, service, role or external system. Separate the component that requests a change from the component authorized to commit it.

State

Question: Which states and transitions can actually exist?

Model normal, edge, recovery and unresolved states. Do not collapse a lost response into failure when the external action may have succeeded.

Boundary

Question: Which component or role is responsible for which behavior?

A boundary should make it possible to say where validation, policy, persistence, side effects and recovery belong. H.A.R.D. does not require microservices or any particular architecture.

Contract

Question: What must be true when information or control crosses a boundary?

Record inputs, outputs, guarantees and meaningful failure semantics at the level required to judge the choice. A contract can be a function, API, event, workflow handoff or human approval rule.

Failure and recovery

Question: What valid or invalid partial state can remain when execution stops?

Record detection, containment, recovery and ownership when relevant. "Retry" is not a complete recovery model when the first attempt may already have succeeded.

Time and ordering

Question: What changes when operations repeat, race, arrive late or complete out of order?

Use this surface when timing affects correctness. Relevant questions can include idempotency, concurrency, stale reads, cancellation races and delayed external responses.

Assumption

Question: What must be true for the current decision to remain reasonable?

Every consequential assumption records:

- the current assumption,
- consequence if it is false,
- evidence needed,
- and the trigger that should reopen it.

An assumption is not made true by writing it down.

Status vocabulary

Each surface is recorded independently:

- **supported:** retained evidence supports the current model within the declared scope;
- **conflicted:** retained evidence contradicts the current model;
- **unassessed:** the reviewer knows what must be inspected or measured next;
- **not_applicable:** the surface does not apply within a recorded boundary and the reason is explicit.

Supported is not certification. Conflicted is not permission for an agent to redesign the product. It means the current decision cannot be treated as settled without a human-owned resolution.

Challenge scenarios

For each deepened choice, define at least one bounded challenge.

Start with:

What would have to be true for this decision to be wrong?

Then record:

1. the condition,
2. the claim at risk,
3. the expected behavior or invariant,
4. the consequence if mishandled,
5. the affected requirement or recovery item,
6. the evidence needed,
7. and the observed result when it has actually been checked.

Useful challenge shapes include, when relevant:

- same request twice,
- two incompatible transitions at the same time,
- delayed first response followed by retry,
- dependency unavailable,
- state persisted but side effect failed,
- side effect succeeded but local state did not,
- cancellation racing completion,
- stale cache or token,
- source-of-truth disagreement,
- authorization context changes,
- expected volume or latency assumption exceeded.

Do not enumerate edge cases for their own sake. Risk and consequence determine depth.

Evidence mapping

Decision-surface reasoning never substitutes for execution evidence.

H.A.R.D. retains the four evidence stages:

Evidence	What it can support
Specified	The declared revision defines the behavior, invariant or acceptance condition.
Walkthrough	A retained scenario walkthrough exercises that defined behavior.

Evidence	What it can support
Implemented	The cited implementation contains the relevant behavior.
Runtime tested	A retained execution checks the exact implementation in the named context.

A supported source-of-truth model may be supported by a specification at specification handoff. It does not imply the implementation enforces it. A passing walkthrough does not become a runtime test.

Changes to consequential choices, their decision surfaces, state context, dependencies, authority or challenge conditions are part of the requirement/context fingerprint. Existing checks cannot be relabeled after those changes.

Smallest coherent next slice

For software work, H.A.R.D. asks for the smallest next slice that can produce useful evidence about the important reasoning.

A coherent slice is not the fewest lines of code and not a requirement to build the whole stack. When applicable, it connects enough of the path to test the important claim:

input -> validation -> state/persistence -> side effect -> failure/recovery -> observable outcome

Only include elements needed by the bounded choice.

Example:

Weak commitment: "Build the invitation dashboard and backend."

Coherent slice: "Prove one authorized invitation from creation through acceptance and persistence, including duplicate accept and revoke/accept ordering, with an observable final membership state."

The slice remains subject to the six H.A.R.D. engineering gates and separate owner authorization.

What this lens must not do

- Do not recover or request private model chain-of-thought.
- Do not claim a newly generated rationale was the creator's original reason.
- Do not automatically recommend queues, retries, microservices, event sourcing or another familiar pattern.
- Do not turn every unknown into a defect.
- Do not infer a race condition, data leak or duplicate side effect without relevant evidence.
- Do not confuse a strong system model with proof that the implementation behaves that way.

- Do not use software-test passes as empirical evidence that this review method improves human decisions.

Example: generated team invitation

Observed result: generated code creates an invitation, emails a token and accepts it into a membership.

Decompressed choices:

- invitation is a separate entity;
- token acceptance creates membership;
- email delivery follows invitation creation.

Relevant surfaces:

- truth: which record establishes membership;
- ownership: which service can grant membership;
- state: pending, accepted, revoked and expired;
- time/ordering: accept versus revoke, repeat acceptance;
- contract: what token verification guarantees;
- assumption: when billing or seat allocation should change.

Challenge:

- condition: revoke and accept occur nearly together;
- claim at risk: revoked invitations cannot grant membership;
- invariant: exactly one valid transition establishes or denies membership according to the authoritative state;
- consequence: unauthorized access or inconsistent billing;
- evidence: specification walkthrough at plan stage, then exact-revision integration evidence after implementation.

H.A.R.D. does not decide in advance whether a database transaction, queue, lock or another mechanism is best. It makes the decision visible enough for an accountable person to choose and verify a mechanism appropriate to the system.